

리눅스 3.1 - 데이터베이스 인덱스

남궁명수

[1] 인덱스

인덱스는 테이블의 레코드를 빠르게 찾기 위해 별도로 만드는 자료구조이다.

책으로 비유하면

- 테이블의 실제 레코드 → 책의 본문
- 인덱스 키 → 책 뒤쪽의 검색 단어
- 레코드 위치 정보 → 해당 단어가 나오는 페이지 번호

다음 테이블이 있다고 해보자.

id	name	age	city
101	철수	25	서울
102	영희	30	부산
103	민수	28	인천
104	지수	25	대전

인덱스가 없다면

```
SELECT * FROM member WHERE age = 28;
```

DBMS는 처음부터 레코드를 확인해야 할 수 있다.

101 확인 → 102 확인 → 103 발견

이를 테이블 풀 스캔(Table Full Scan)이라고 한다.

age에 인덱스가 있다면:

```
CREATE INDEX idx_member_age ON member(age);
```

DBMS는 정렬된 인덱스에서 28을 찾은 다음 해당 레코드로 접근한다.

age 인덱스에서 28 검색

↓

레코드를 찾기 위한 정보 획득

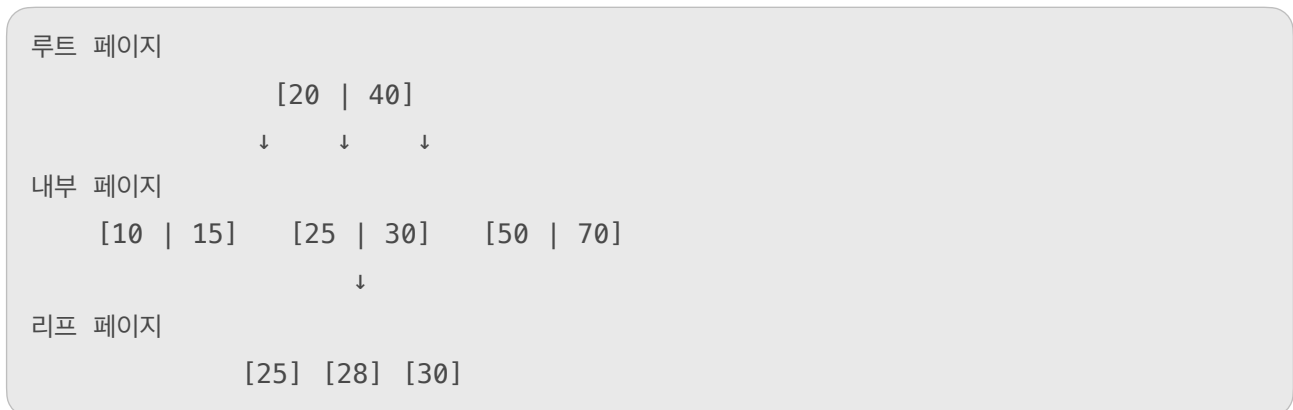
↓

실제 레코드 접근

[2] 인덱스의 기본 구조

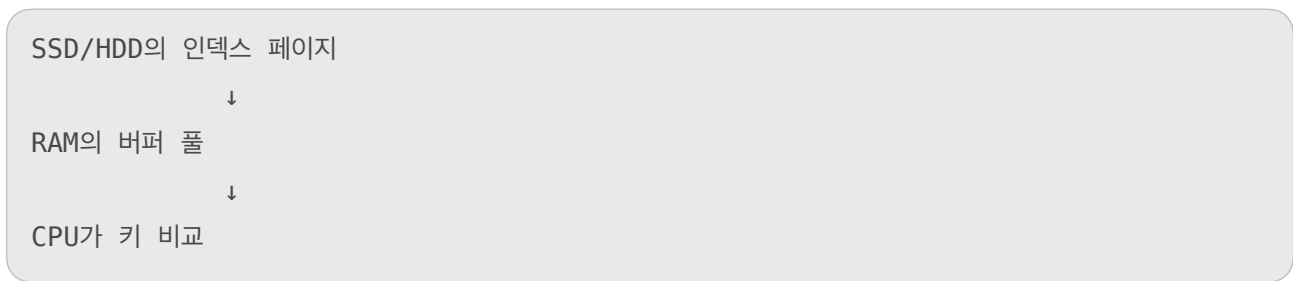
일반적인 관계형 데이터베이스 인덱스는 B+Tree 계열 자료구조를 많이 사용한다.

인덱스도 데이터 파일 안에서 데이터베이스 페이지 단위로 저장된다.



각 노드는 메모리 속의 작은 구조체가 아니라, 보통 데이터베이스 파일의 페이지에 저장되는 구조다.

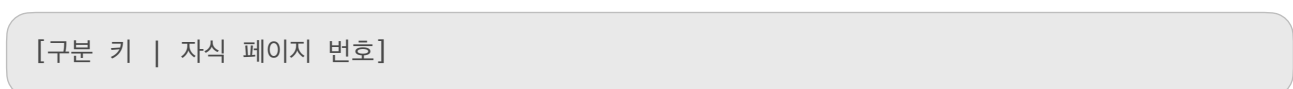
조회할 때 필요한 인덱스 페이지가 RAM의 버퍼 풀에 없다면 저장장치에서 읽어온다.



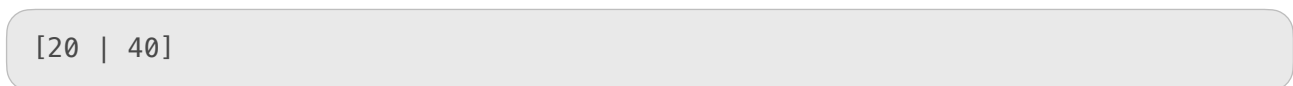
[3] B+Tree 내부 노드와 리프 노드

[내부 노드]

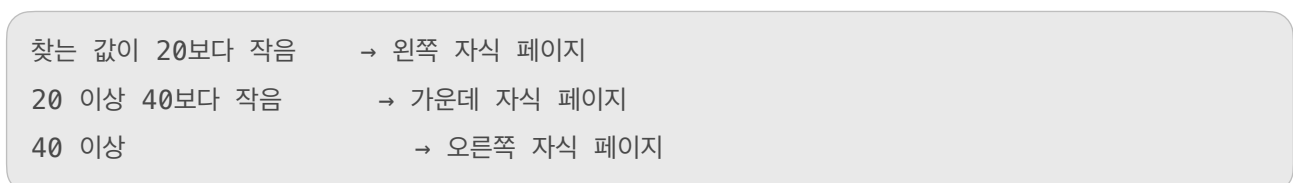
내부 노드에는 실제 레코드 전체가 아니라 다음과 같은 정보가 들어간다.



예를 들어,



이 키들은 탐색 방향을 결정한다.



내부 노드의 키는 실제 데이터를 반환하기 위한 값이 아니라, 탐색 범위를 나누는 기준이다.

[리프 노드]

리프 노드에는 인덱스 종류에 따라 다음 중 하나가 들어간다.

- 인덱스 키와 실제 레코드
- 인덱스 키와 기본키
- 인덱스 키와 레코드 위치 정보

내부 노드: 어디로 내려갈지 결정

리프 노드: 검색 결과 또는 레코드 위치를 제공

[4] 클러스터형 인덱스

클러스터형 인덱스(Clustered Index)는 B+Tree 리프 페이지 자체에 실제 레코드가 저장되는 형태다.

MySQL InnoDB에서는 일반적으로 기본키가 클러스터형 인덱스가 된다.

다음과 같은 테이블이 있다고 해보자.

```
CREATE TABLE member (  
    id    INT PRIMARY KEY,  
    name VARCHAR(20),  
    age   INT  
);
```

기본키 B+Tree는 개념적으로 다음과 같다.

루트 페이지
[100 | 200]

↓

리프 페이지
[101 | 철수 | 25]
[102 | 영희 | 30]
[103 | 민수 | 28]

리프 페이지에서 기본키를 찾으면 실제 레코드도 함께 찾게 된다.

```
SELECT * FROM member WHERE id = 103;
```

조회 과정은 다음과 같다.

기본키 B+Tree 탐색

↓

리프 페이지에서 id=103 발견

↓

같은 위치에 저장된 실제 레코드 반환

[103 | 민수 | 28]

[클러스터형이라는 말의 의미]

레코드가 클러스터형 인덱스 키 순서에 따라 논리적으로 정렬되어 리프 페이지에 저장된다는 뜻이다.

페이지 A		페이지 B
101 철수		104 지수
102 영희	→	105 준호
103 민수		106 수진

하지만 모든 레코드가 SSD의 물리적 공간에서 완전히 연속되어 있다는 뜻은 아니다. 페이지들은 논리적인 키 순서를 유지하지만, 실제 저장장치에서는 서로 떨어진 위치에 존재할 수도 있다.

[클러스터형 인덱스의 특징]

- 리프 페이지에 실제 레코드가 들어 있다.
- 키 순서에 따라 레코드가 논리적으로 배치된다.
- 한 테이블의 레코드 배치 기준은 하나뿐이므로 일반적으로 하나만 존재한다.
- 기본키 검색과 범위 검색에 유리하다.
- 중간 위치에 레코드가 삽입되면 페이지 분할이 발생할 수 있다.

[5] 보조 인덱스

보조 인덱스(Secundary Index)는 클러스터형 인덱스 이외에 추가로 생성하는 인덱스다. 비클러스터형 인덱스(Non-clustered Index)라고 부르기도 하지만, DBMS마다 용어와 내부 구현에는 차이가 있다.

```
CREATE INDEX idx_member_age ON member(age);
```

MySQL InnoDB의 보조 인덱스 리프 페이지에는 일반적으로 다음 정보가 저장된다.

[보조 인덱스 키 | 기본키]

예를 들어,

```
[25 | 101]
[25 | 104]
[28 | 103]
[30 | 102]
```

여기에는 name과 같은 실제 레코드 전체가 들어있지 않다.

[보조 인덱스를 통한 조회]

```
SELECT * FROM member WHERE age = 28;
```

조회 과정은 다음과 같다.

1. age 보조 인덱스 탐색
2. 리프 페이지에서 [28 | 기본키 103] 발견
3. 기본키 103으로 클러스터형 인덱스 재탐색
4. [103 | 민수 | 28] 실제 레코드 반환

보조 인덱스에서 기본키 인덱스로 다시 찾아가는 과정을 흔히 테이블 룩업 또는 InnoDB 문맥에서 클러스터형 인덱스 룩업이라고 이해할 수 있다.

[6] 기본키 인덱스

기본키 인덱스(Primar Key Index)는 기본키 제약조건으로 만들어진 인덱스다.

```
CREATE TABLE member (  
    id INT PRIMARY KEY  
);
```

기본키에는 다음 조건이 들어 있다.

- 값이 중복될 수 없다.
- NULL일 수 없다.
- 각 레코드를 고유하게 식별한다.

MySQL InnoDB에서는 기본키 인덱스가 클러스터형 인덱스가 된다.
하지만 다음 두 용어가 항상 완전히 같은 뜻은 아니다.

- 기본키 인덱스 -> 기본키 제약조건으로 만든 인덱스
- 클러스터형 인덱스 -> 실제 레코드의 저장 구조를 결정하는 인덱스

DBMS에 따라 클러스터형 인덱스를 기본키가 아닌 열에 만들 수도 있다.

[7] 유니크 인덱스

유니크 인덱스(Unique Index)는 인덱스 키의 중복을 허용하지 않는다.

```
CREATE UNIQUE INDEX idx_member_email  
ON member(email);
```

```
a@test.com → 가능  
b@test.com → 가능  
a@test.com → 중복이므로 거부
```

유니크 인덱스는 두 가지 역할을 한다.

- 빠른 검색
- 중복 값 방지

기본키와 비슷하지만 차이가 있다.

구분	기본키	유니크 인덱스
중복	불가능	불가능
NULL	불가능	DBMS 규칙에 따라 허용 가능
테이블당 개수	하나	여러 개 가능
주요 목적	레코드 식별	값의 고유성 보장

[8] 단일 열 인덱스

하나의 열로 만든 인덱스다.

```
CREATE INDEX idx_member_age
ON member(age);
```

인덱스 키:

```
age
```

다음 조건에 활용될 수 있다.

```
WHERE age = 28
WHERE age >= 20
ORDER BY age
```

[9] 복합 인덱스

복합 인덱스(Composite Index)는 둘 이상의 열을 묶어서 만든 인덱스다.

```
CREATE INDEX idx_member_city_age
ON member(city, age);
```

인덱스 키는 다음 순서로 정렬된다.

```
(city, age)
```

```
(대전, 25)
```

```
(부산, 30)
```

```
(서울, 20)
```

먼저 city를 정렬하고, 같은 city 안에서 age로 정렬한다.

[왼쪽 우선 규칙]

복합 인덱스 (city, age)는 일반적으로 왼쪽 열부터 사용해야 효과적으로 탐색할 수 있다.

```
-- 활용 가능
WHERE city = '서울'

-- 활용 가능
WHERE city = '서울' AND age = 28

-- 일반적으로 city 없이 age만으로는 전체 인덱스를 효율적으로 탐색하기 어려움
WHERE age = 28
```

전화번호부가 다음 순서로 정렬된 것 같다.

성 → 이름

성을 모른 채 이름만 가지고 찾으면 정렬 순서를 충분히 활용하기 어렵다.

[10] 커버링 인덱스

쿼리에 필요한 모든 열이 인덱스에 들어 있으면 실제 레코드로 다시 찾아가지 않아도 된다.
이를 커버링 인덱스라고 한다.

다음 복합 인덱스가 있다고 해보자.

```
CREATE INDEX idx_member_age_name
ON member(age, name);
```

쿼리:

```
SELECT name
FROM member
WHERE age = 28;
```

필요한 값은 모두 인덱스에 있다.

- 검색 조건: age
- 반환 데이터: name

따라서 다음처럼 처리할 수 있다.

```
보조 인덱스에서 age=28 검색
      ↓
같은 인덱스에서 name 확인
      ↓
종료
```

기본키 인덱스로 다시 이동할 필요가 없다.
주의할 점은 커버링 인덱스가 별도의 인덱스 종류라기보다는 다음 상태를 의미한다는 것이다.

특정 쿼리에 필요한 데이터가 전부 인덱스에 포함된 상태

[11] 범위 검색에서 인덱스가 유리한 이유

B+Tree의 리프 페이지는 키 순서대로 정렬되고 서로 연결되어 있다.

리프 페이지 A 리프 페이지 B
[20] [21] [22] → [23] [24] [25]

다음 쿼리를 실행한다고 하자.

```
SELECT *  
FROM member  
WHERE age BETWEEN 20 AND 25;
```

DBMS는 먼저 20의 위치를 찾은 후 리프 페이지를 순서대로 읽을 수 있다.

20 찾기
↓
20 → 21 → 22 → 23 → 24 → 25

따라서 B+Tree 인덱스는 다음 연산에 적합하다.

- 일치 검색
- 범위 검색
- 정렬
- 최솟값과 최댓값
- 접두사 조건

[12] 인덱스와 RAM의 관계

인덱스도 저장장치에 영구적으로 저장되는 데이터다.

SSD/HDD
├─ 테이블 데이터 페이지
├─ 기본키 인덱스 페이지
└─ 보조 인덱스 페이지

조회할 때는 필요한 인덱스 페이지를 RAM의 버퍼 풀로 가져온다.

1. 루트 페이지가 버퍼 풀에 있는지 확인
2. 없으면 저장장치에서 읽기
3. 자식 페이지 번호 확인
4. 자식 페이지가 버퍼 풀에 있는지 확인
5. 리프 페이지까지 반복
6. 실제 레코드 반환

자주 사용되는 루트와 상위 내부 페이지는 버퍼 풀에 남아 있을 가능성이 높다.
따라서 실제 조회에서는 모든 단계마다 저장장치 I/O가 발생하는 것은 아니다.

루트 페이지	→ RAM에 있음
내부 페이지	→ RAM에 있음
리프 페이지	→ RAM에 없음
	↓
	저장장치 I/O 발생

B+Tree의 높이가 낮은것도 중요하지만, 필요한 페이지가 버퍼 풀에 있는지도 실제 성능에 큰 영향을 준다.

[13] 인덱스가 항상 빠른 것은 아니다

인덱스에는 비용도 있다.

[저장 공간 사용]

인덱스도 페이지로 저장되므로 저장 공간을 차지한다.

테이블 데이터 1GB
+ 기본키 인덱스
+ age 인덱스
+ name 인덱스
+ city_age 인덱스

[쓰기 성능 감소]

레코드를 삽입하거나 수정하면 관련 인덱스도 함께 변경해야 한다.

```
INSERT INTO member ...
```

처리해야 하는 작업:

실제 레코드 삽입
기본키 인덱스 갱신
age 인덱스 갱신
name 인덱스 갱신

인덱스가 많을수록, INSERT, UPDATE, DELETE 비용이 증가할 수 있다.

[낮은 선택도]

서로 다른 값의 종류가 적은 열은 인덱스 효과가 작을 수 있다.

성별: 남/여

활성 상태: true/false

예를 들어, 전체 100만 건 중 80만 건이 active = true라면, 인덱스를 거쳐 80만 건을 찾는 것보다 테이블을 순차적으로 읽는 편이 유리할 수도 있다.

그래서 인덱스가 있어도 옵티마이저가 풀 스캔을 선택할 수 있다.

[14] 전체 구조 정리

MySQL InnoDB를 기준으로 단순화하면 다음과 같다.

테이블

- |
 - ├─ 클러스터형 인덱스
 - | ├─ 키: 기본키
 - | └─ 리프 페이지: 실제 레코드 전체
- |
 - ├─ 보조 인덱스 age
 - | ├─ 키: age
 - | └─ 리프 페이지: age + 기본키
- |
 - └─ 보조 인덱스 name
 - | ├─ 키: name
 - | └─ 리프 페이지: name + 기본키

조회 관계는 다음과 같다.

기본키 조회

기본키 인덱스 → 실제 레코드

보조 인덱스 조회

보조 인덱스 → 기본키 → 기본키 인덱스 → 실제 레코드

커버링 인덱스 조회

보조 인덱스 → 필요한 값 반환

- 인덱스는 정렬된 키를 이용해 실제 레코드가 있는 위치를 빠르게 찾는 자료구조다.
- 클러스터형 인덱스는 리프 페이지에 실제 레코드를 저장하고, 보조 인덱스는 일반적으로 인덱스 키와 실제 레코드를 찾기 위한 정보를 저장한다.
- MySQL InnoDB에서는 기본키가 인덱스이며, 보조 인덱스의 리프 페이지에는 보조 인덱스 키와 기본키가 저장된다.
- 인덱스 역시 SSD/HDD에 페이지 단위로 저장되며, 조회할 때 필요한 페이지가 RAM의 버퍼 풀로 올라온 후 CPU가 처리한다.